

Exercises 08

Query Optimisation and Execution

[\[Show with no answers\]](#) [\[Show with all answers\]](#)

1. Consider the following tables relating to trips on a suburban bus network

```
Trip(fromPlace:integer, toPlace:integer, tripDate:date)
Place(placeId:integer, address:string, suburb:string)
```

a. Write an SQL query that returns all of the addresses in Randwick that are the destination of a trip on March 4, 2005.

[\[show answer\]](#)

b. Give a naive translation of the SQL query into a relational algebra expression.

[\[show answer\]](#)

c. Translate the naive relational algebra expression into an equivalent expression using pushing of selections and projections.

[\[show answer\]](#)

d. Translate the optimized relational algebra expression into the most directly corresponding SQL query.

[\[show answer\]](#)

2. What are the possible join trees (without cross-products) for each of the following queries:

a. `select * from R,S,T where R.a=S.b and S.c=T.d`

[\[show answer\]](#)

b. `select * from R,S,T where R.a=S.b and T.c=R.d`

[\[show answer\]](#)

c. `select * from R,S,T where R.a=S.b and S.c=T.d and T.e=R.f`

[\[show answer\]](#)

d. `select * from R,S,T,U`
`where R.a=S.b and S.c=T.d and T.e=R.f and T.g=U.h and S.i=U.j`

[\[show answer\]](#)

Do not include trees/sub-trees that are reflections of other tree/subtrees.



3. Consider a table $R(a,b,c)$ and assume that

- all attributes have uniform distribution of data values
- attributes are independent of each other
- all attributes are **NOT NULL**
- $r = 1000$, $V(a,R) = 10$, $V(b,R) = 100$

Calculate the expected number of tuples in the result of each of the following queries:

- `select * from R where not a=k`
- `select * from R where a=k and b=j`
- `select * from R where a in (k,l,m,n)`

where j, k, l, m, n are constants.

[\[show answer\]](#)

4. Consider the following tables relating to retail sales:

```
create table Item (
    iname      text,
    category   text,
    primary key (name)
);
create table Store (
    sname      text,
    city       text,
    street     text,
    primary key (city,street)
);
create table Transaction (
    item       text references Item(iname),
    store      text references Store(sname),
    tdate      date,
    primary key (item,store,tdate)
);
```

Consider the following query (expressed as SQL and relational algebra):

```
select category,city
from   Item, Store, Transaction
where  iname=item and store=sname and
       tdate='20-12-2004' and city='Sydney';
```

```
JoinResult  = Item Join[iname=item] Transaction Join[store=sname] St
SelectResult = Sel[tdate='20-12-2004' and city='Sydney'](JoinResult)
```



```
FinalResult = Proj[category,city](SelectResult)
```

Show the three “most promising” relational algebra expressions that the query optimizer is likely to consider; then find the most efficient query plan and estimate its cost.

Assume 50 buffer pages and the following statistics and indices:

- **Item**: 50,000 tuples, 10 tuples/page.
Indexing: hashed on **iname** (assume no overflows).
- **Store**: 1,000 tuples, 5 tuples/page; 100 cities.
Index1: Unclustered hash index on **sname**. Index2: Clustered 2-level B+tree on **city**.
- **Transaction**: 500,000 tuples, 25 tuples/page; 10 items bought per store per day.
The relation stores transactions committed over a 50 day period.
Index: 2-level clustered B+tree on the pair of attributes **store, ttime**.

[\[show answer\]](#)

